

Übung 4 - Unabhängige Zufallsereignisse

Michael Geuze

A-1)

Unter der Voraussetzung der Poisson-Verteilung $P(k|\lambda) = \frac{\lambda^k}{k!} \cdot e^{-\lambda}$ und der Annahme, die Verteilung ist in die Zeiteinheit Viertelstunde aufgeteilt folgt daraus, dass das Ereignis pro Zeiteinheit

$\lambda = 7/4$

mal vorkommt. Also ist $\lambda = 7/4 = 1.75$. Daraus folgt, dass das Ereignis in einer Viertelstunde nie vorkommt gleich $P(0|\lambda) = e^{-\lambda} \approx 0.1738 = 17.38\%$ ist.

$P = \exp(-\lambda)$

A-2)

Ausgangsformel:

$$P(k|\lambda) = \frac{\lambda^k}{k!} \cdot e^{-\lambda}$$

Im Zeitintervall τ :

$$P(k|\tau \cdot \lambda) = \frac{(\tau \cdot \lambda)^k}{k!} \cdot e^{-\tau \cdot \lambda}$$

$$P(k=0|\tau \cdot \lambda) = \frac{(\tau \cdot \lambda)^0}{0!} \cdot e^{-\tau \cdot \lambda} = e^{-\tau \cdot \lambda}$$

Die gegebene Formel beschreibt die Wahrscheinlichkeit dafür, dass in einem Zeitraum τ kein Ereignis eintritt, das gemäß der Poisson-Verteilung mit einer erwarteten Rate von λ stattfindet.

A-3)

In diesem Fall repräsentiert die Zufallsvariable X die Wartezeit bis zum Eintreten eines Ereignisses.

$$P(X > t|\lambda) = e^{-\lambda \cdot t}$$

A-4)

$$P_{exp}(X \leq t|\lambda) = 1 - P(X > t|\lambda) = 1 - e^{-\lambda \cdot t}$$

A-5)

Wenn man $P(X = t|\lambda)$ betrachtet ergibt dies keinen Sinn. Die Zufallsvariable X ist eine kontinuierliche Variable. Daher müssen wir nach der Wahrscheinlichkeit in einem Intervall fragen.

$$\begin{aligned} P(t \leq X \leq t + \epsilon) &= P(X \leq t + \epsilon) - P(X \leq t) = 1 - e^{-\lambda \cdot (t+\epsilon)} - (1 - e^{-\lambda \cdot t}) = e^{-\lambda \cdot t} - e^{-\lambda \cdot (t+\epsilon)} \\ &= e^{-\lambda \cdot t} (1 - e^{-\lambda \cdot \epsilon}) = \end{aligned}$$

mit Taylorreihenentwicklung

$$= \lambda \cdot \epsilon \cdot e^{-\lambda \cdot t} \left(1 - \frac{\lambda}{2!} \epsilon + \frac{\lambda^2}{3!} \epsilon^2 - \frac{\lambda^3}{4!} \epsilon^3 + \dots \right)$$

Dann ist die Änderungsrate zum Zeitpunkt t gleich:

$$\begin{aligned} \frac{dP(X = t|\lambda)}{dt} &:= \lim_{\epsilon \rightarrow 0} \frac{(P(X \leq t + \epsilon|\lambda) - P(X \leq t|\lambda))}{\epsilon} = \lim_{\epsilon \rightarrow 0} \frac{P(t \leq X \leq t + \epsilon)}{\epsilon} \\ \frac{dP(X = t|\lambda)}{dt} &= \lambda \cdot e^{-\lambda \cdot t} \end{aligned}$$

Nach umformen erhält man:

$$dP(X = t|\lambda) = \lambda \cdot e^{-\lambda \cdot t} \cdot dt$$

Und ergibt als Wahrscheinlichkeitsdichte der Exponentialfunktion:

$$f_{exp}(t|\lambda) = \lambda \cdot e^{-\lambda \cdot t}$$

B-1)

Zu schätzen ist bei den gegebenen Zahlen schwierig, da sie alle durch 40 Teilbar sind. 40 ist auch Teiler der Summe 4000 und exakt 1%. Daher "schätzen" sich folgende Werte (die mit äußerst hoher Wahrscheinlichkeit an die echten Werte grenzen):

$$3000 \pm 500 = 4\%$$

$$3500 \pm 500 = 6\%$$

$$4000 \pm 500 = 10\%$$

$$4500 \pm 500 = 15\%$$

$$5000 \pm 500 = 30\%$$

$$5500 \pm 500 = 20\%$$

$$6000 \pm 500 = 8\%$$

$$6500 \pm 500 = 5\%$$

$$7000 \pm 500 = 2\%$$

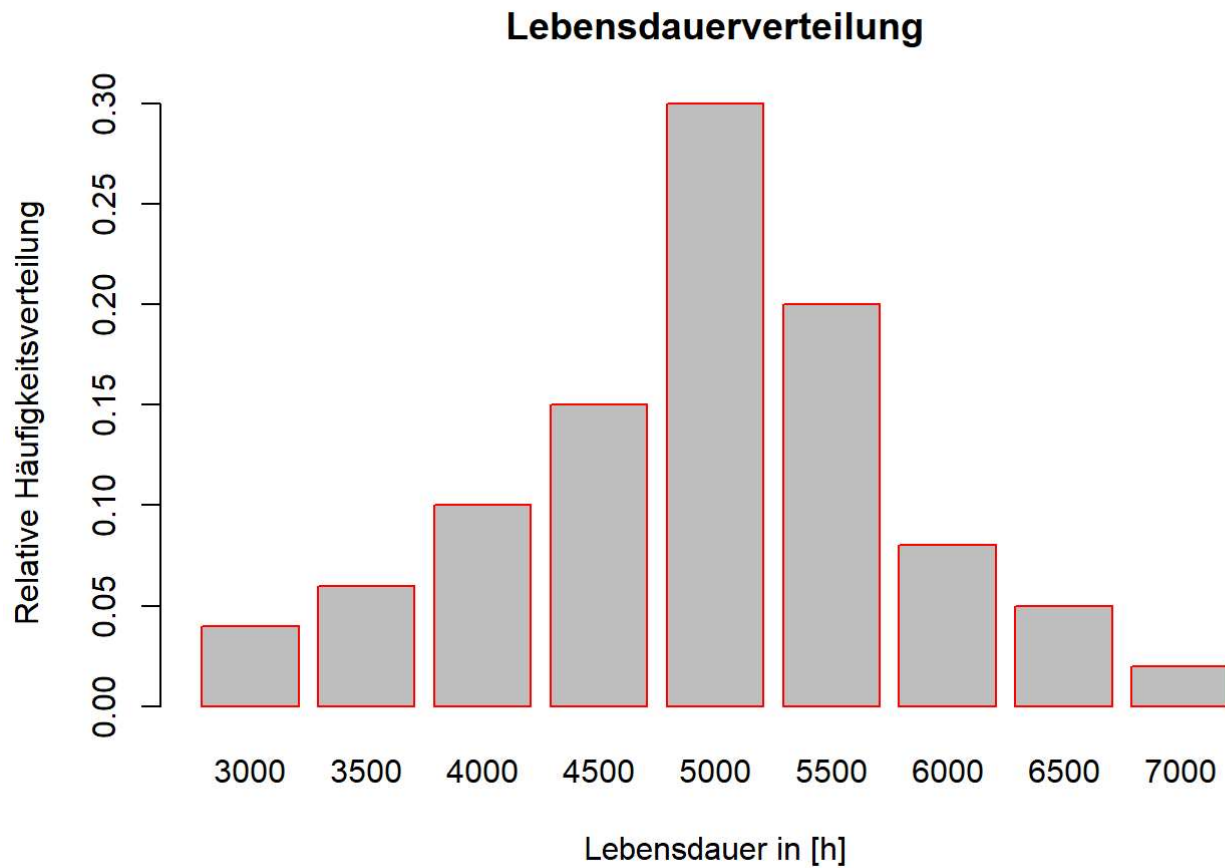
```
# Lifetime mids
lt_mids.vec = seq(3000, 7000, 500)
# Lifetime class broadness
lt_class_broadness.vec = rep(500, 9)
lt_class_breaks.vec = seq(2750, 7250, 500)
# Lifetime absolute occurrences
lt_class_abs_occ.vec = c(160, 240, 400, 600, 1200, 800, 320, 200, 80)
lt_class_abs_occ = sum(lt_class_abs_occ.vec)
lt_class_rel_occ.vec = lt_class_abs_occ.vec / lt_class_abs_occ
# Calculate mean value
lt_mean = sum(lt_mids.vec * lt_class_abs_occ.vec) / lt_class_abs_occ
# Calculate standard deviation, non empiric, as n >>
lt_std_dev = sqrt(sum(((lt_mean - lt_mids.vec)^2) * (lt_class_abs_occ.vec / lt_class_abs_occ)))
cat("Mittelwert: ", lt_mean)
```

```
## Mittelwert: 4950
```

```
cat("Standardabweichung: ", lt_std_dev)
```

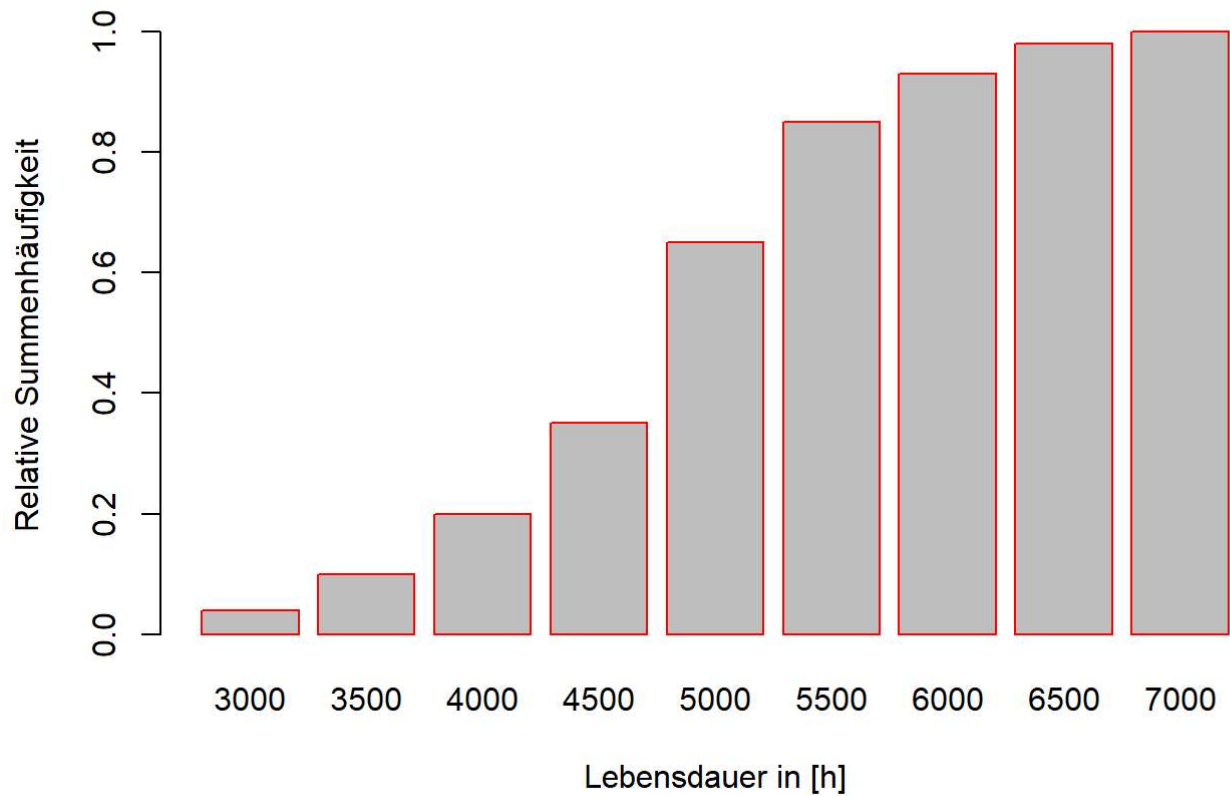
```
## Standardabweichung: 867.4676
```

```
barplot(lt_class_rel_occ.vec,
        names.arg = lt_mids.vec,
        xlab = "Lebensdauer in [h]",
        ylab = "Relative Häufigkeitsverteilung ",
        main = "Lebensdauerverteilung",
        border = "red")
```



```
b = barplot(cumsum(lt_class_rel_occ.vec),  
            names.arg = lt_mids.vec,  
            xlab = "Lebensdauer in [h]",  
            ylab = "Relative Summenhäufigkeit",  
            main = "Lebensdauerverteilung Summenhäufigkeit",  
            border = "red")
```

Lebensdauerverteilung Summenhäufigkeit

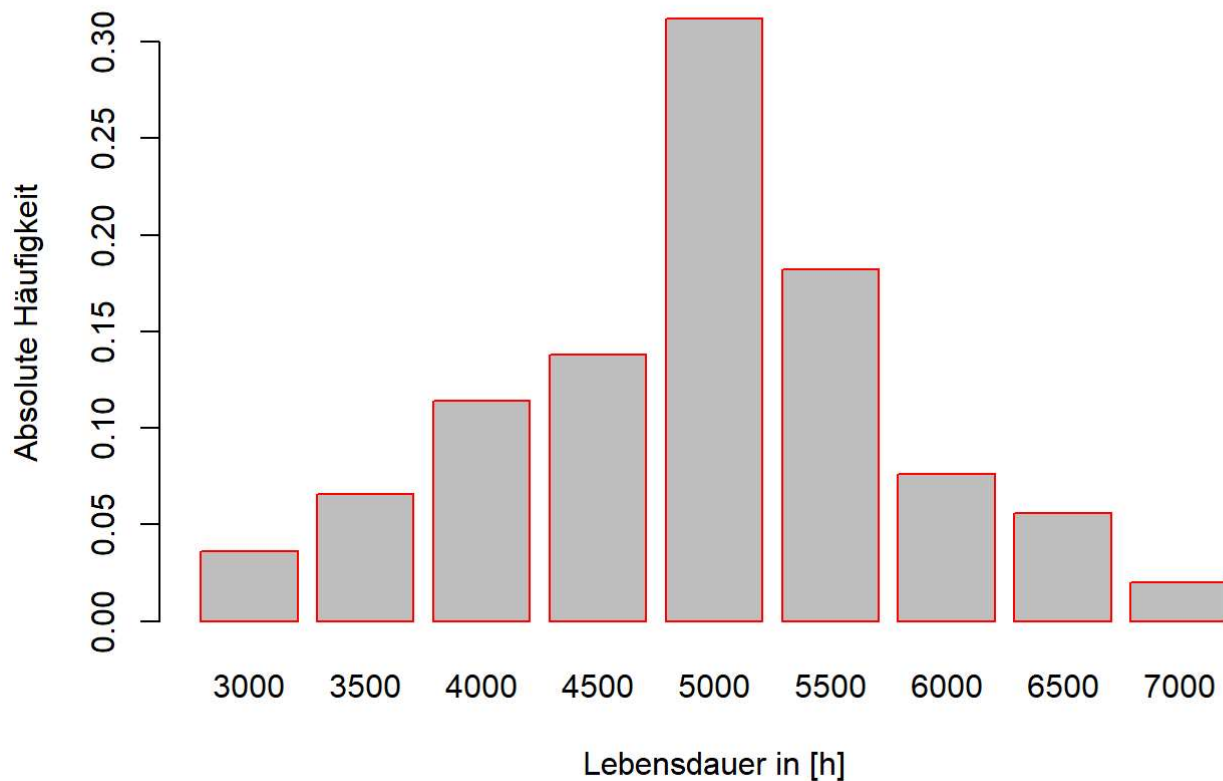


B-2)

```
lt_sim_classes.tab = table(cut(runif(500), breaks = c(0, cumsum(lt_class_rel_occ.vec)) , right = TRUE))
```

```
barplot(lt_sim_classes.tab / 500,
        xlab = "Lebensdauer in [h]",
        ylab = "Absolute Häufigkeit",
        main = "Lebensdauerverteilung, simuliert",
        names.arg = lt_mids.vec,
        border = "red")
```

Lebensdauerverteilung, simuliert



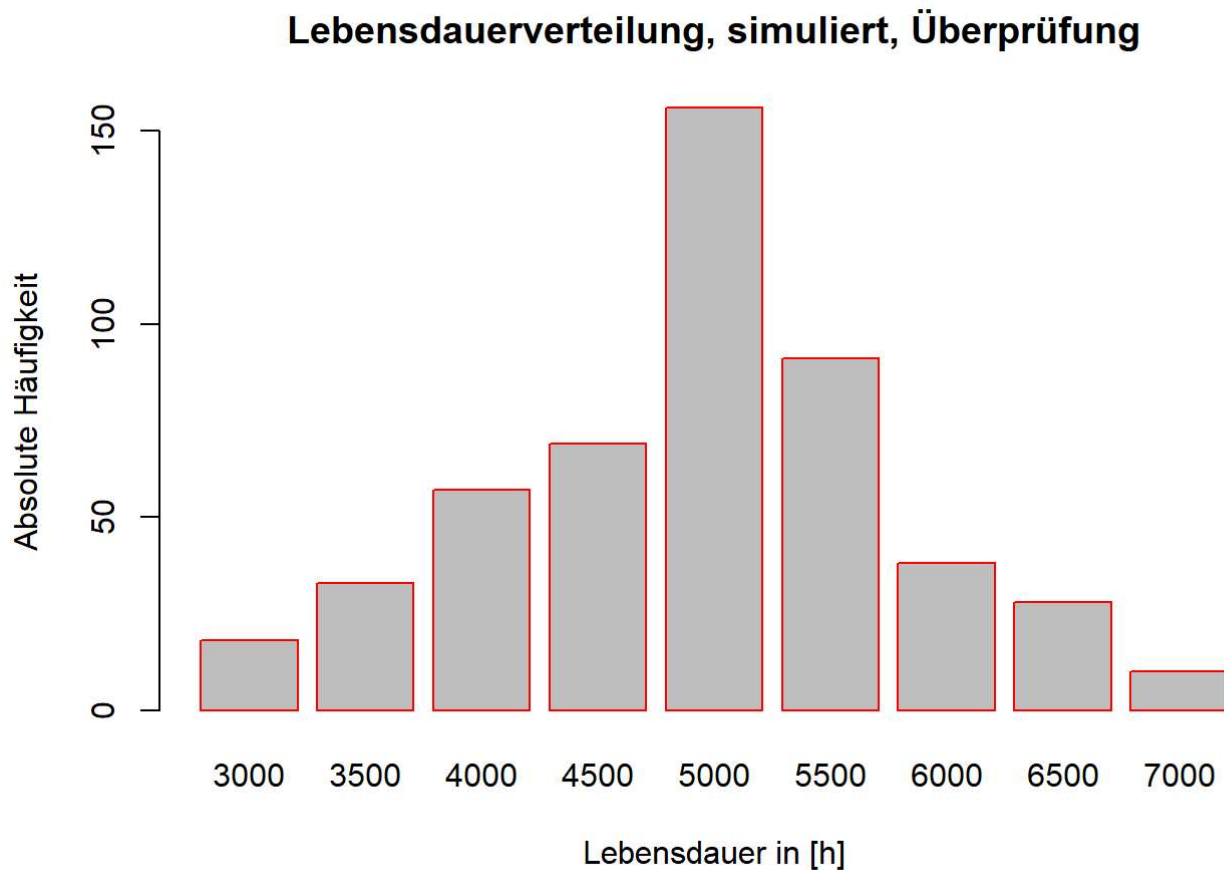
```
# Create a table with uniform distributed numbers with count of lt_simulated per bin in the c
lasses
lt_simulated = c()
for (i in 1:9){
  # Append the uniform distributed values n times to the vector lt_simulated
  lt_simulated = c(lt_simulated,
    runif(
      n = lt_sim_classes.tab[[i]],
      min = (lt_mids.vec[i]) - lt_class_broadness.vec[i] / 2,
      max = (lt_mids.vec[i]) + lt_class_broadness.vec[i] / 2
    )
  )
}
# Crosscheck for standard deviation and mean
mean(lt_simulated)
```

```
## [1] 4937.623
```

```
sd(lt_simulated)
```

```
## [1] 878.238
```

```
lt_simulated.tab = table(cut(lt_simulated, breaks = lt_class_breaks.vec))  
# Crosscheck the new generated data  
barplot(lt_simulated.tab,  
        xlab = "Lebensdauer in [h]",  
        ylab = "Absolute Häufigkeit",  
        main = "Lebensdauerverteilung, simuliert, Überprüfung",  
        names.arg = lt_mids.vec,  
        border = "red")
```



B-3)

```
Ka = 18600
Kv = 6400
Tsim = 7250
tau.vec = seq(500, Tsim, 200)
Kges = c()

for (j in 1:length(tau.vec)){
  tau = tau.vec[j]

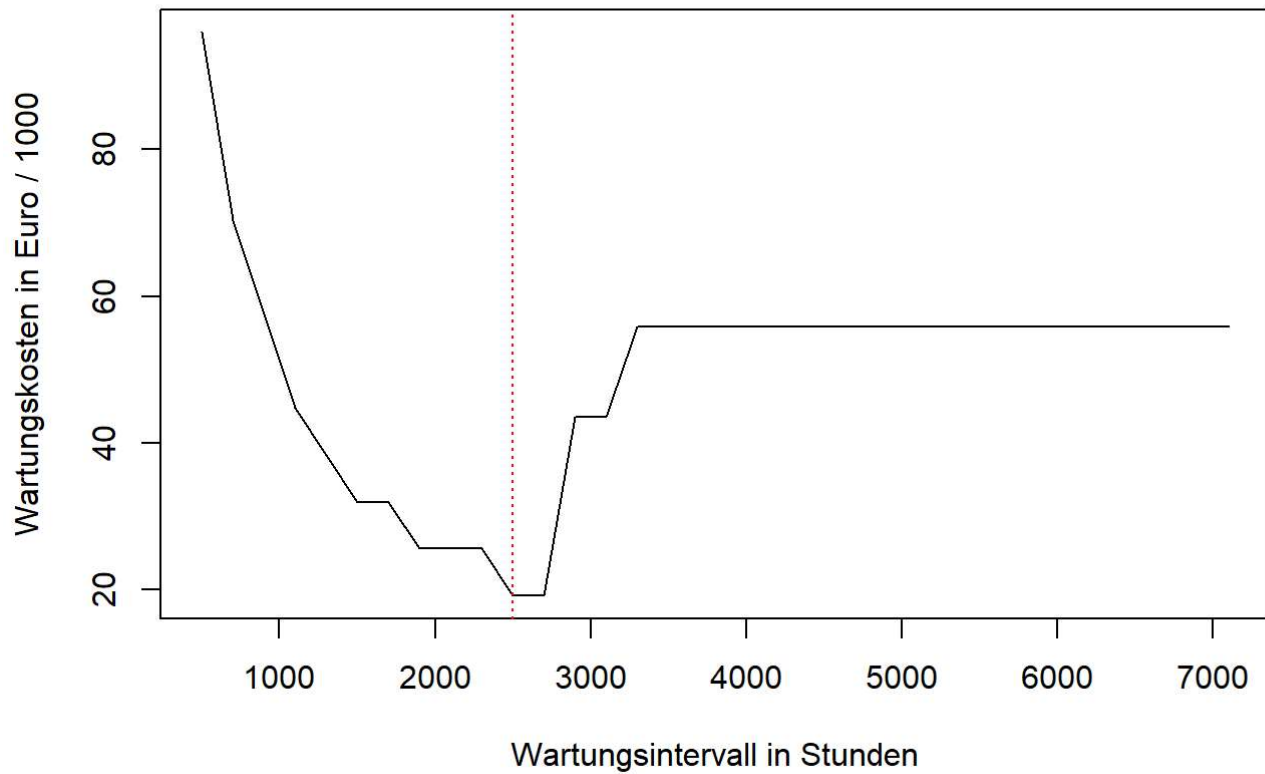
  i = 1
  ti = 0
  za = 0
  zv = 0

  while(TRUE){
    xi <- lt_simulated[i]
    if (xi > tau){
      ti = ti + tau
      zv = zv + 1
    }else{
      ti = ti + xi
      za = za + 1
    }
    if(ti > Tsim || i >= length(lt_simulated)){
      break
    }
    i = i + 1
  }
  Kges[j] = Ka*za + Kv*zv
}

min = min(Kges)

plot(tau.vec, Kges/1000, type="l",
     main = "Wartungskosten",
     xlab = "Wartungsintervall in Stunden",
     ylab = "Wartungskosten in Euro / 1000"
     )
min_tau = tau.vec[Kges == min][1]
abline(v=min_tau, col="red", lty = 3)
```


Wartungskosten



```
cat("Minimale kosten sind: ", min, "Euro")
```

```
## Minimale kosten sind: 19200 Euro
```

```
cat("Bei ", min_tau, " Stunden")
```

```
## Bei 2500 Stunden
```